

LangChain 을 활용한 지능형 에이전트

대화형 AI 시스템 구축

제 1 강: 대화형 AI 아키텍처의 진화와 LangChain 프레임워크의 기초

1.1 현대적 AI 개발 방법론과 에이전트의 정의

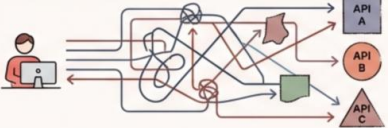
현대 소프트웨어 공학에서 대화형 인공지능(AI)의 개발 패러다임은 단순한 스크립트 기반의 챗봇에서 벗어나, 추론(Reasoning)과 행동(Action)이 결합된 '에이전트(Agent)' 시스템으로 급격히 진화하고 있습니다. 이러한 변화를 주도하는 핵심 프레임워크인 랭체인(LangChain)의 기술적 아키텍처를 심도 있게 분석합니다.

과거의 챗봇 개발 방식은 OpenAI 의 GPT API 와 같은 언어 모델(LLM)의 엔드포인트에 직접 HTTP 요청을 보내는 방식이 주를 이루었습니다. 이러한 방식은 초기 프로토타이핑에는 유리할 수 있으나, 시스템이 복잡해질수록 유연성과 확장성 면에서 치명적인 한계를 드러냅니다. 예를 들어, 모델을 GPT-4 에서 Claude 3 나 Gemini 로 교체해야 할 경우, API 호출 시그니처가 다르기 때문에 코드베이스 전반에 걸친 대대적인 수정이 불가피해집니다.

랭체인은 이러한 문제를 해결하기 위해 등장한 오케스트레이션 프레임워크입니다. 이 프레임워크는 LLM 기반 애플리케이션 개발에 필요한 공통적인 패턴들을 추상화하여 제공함으로써, 개발자가 모델의 내부 통신 로직보다는 비즈니스 로직과 데이터 흐름에 집중할 수 있도록 돕습니다. 특히 랭체인은 요약, 검색(Retrieval), 문서 생성, 질의응답(Q&A)과 같은 다양한 기능을 모듈 단위로 제공하며, 이를 레고 블록처럼 조립하여 복잡한 파이프라인을 구축할 수 있게 합니다.

AI 개발의 패러다임 전환: Raw API에서 오케스트레이션으로

과거의 직접적인 LLM API 호출 방식은 모델 교체나 기능 확장에 한계가 명확합니다. LangChain은 이러한 문제를 해결하는 '오케스트레이션 프레임워크'로서, 개발자가 비즈니스 로직에 집중 할 수 있는 표준 아키텍처를 제시합니다.

직접 API 호출	LangChain 프레임워크
	
추상화 수준: - 낮음	추상화 수준: ✔ 높음
모델 교체 용이성: - 어려움	모델 교체 용이성: ✔ 매우 쉬움
기능 확장성: - 제한적	기능 확장성: ✔ 우수함 (with built-in modules)
메시지 구조: - 비정형 딕셔너리	메시지 구조: ✔ 객체 지향 메시지 클래스
유지 보수성: - 복잡도 증가	유지 보수성: ✔ 모듈화로 유리

LangChain은 단순한 라이브러리가 아니라, 복잡한 LLM 애플리케이션을 위한 현대적인 개발 방법론입니다.

Langchain install

```
(venv) > pip install langchain
(venv) > pip install langchain-openai
(venv) > pip install ipywidgets
```

표 1.1: 직접 API 호출 방식과 랭체인 프레임워크 방식의 비교 분석

비교 항목	직접 API 호출 (Raw API)	랭체인 프레임워크 (LangChain)
추상화 수준	낮음 (Low-level). HTTP 요청 및 JSON 파싱을 직접 구현해야 함.	높음 (High-level). 통일된 인터페이스로 다양한 모델을 제어.
모델 교체 용이성	어려움. 모델별로 다른 API 명세에 맞춰 코드를 수정해야 함.	매우 쉬움. 모델 선언 부분(ChatOpenAI 등)만 변경하면 됨.
기능 확장성	제한적. 검색(RAG)이나 메모리 기능을 직접 구현해야 함.	우수함. 내장된 모듈(Retrievers, Memory)을 즉시 활용 가능.
메시지 구조	딕셔너리 형태의 비정형 데이터 ({ <code>"role": "user", ...</code> }).	객체 지향적 메시지 클래스 (HumanMessage, AIMessage).
유지 보수성	코드 복잡도 증가로 인해 유지 보수가 어려움.	모듈화된 구조로 인해 유지 보수 및 협업에 유리함.

이러한 비교 분석을 통해 알 수 있듯이, 랭체인은 단순한 라이브러리를 넘어 LLM 애플리케이션의 표준 아키텍처를 제시하고 있습니다. 본 강의에서는 이러한 랭체인의 철학을 바탕으로, 단순한 챗봇을 넘어 외부 도구를 스스로 선택하고 실행하는 에이전트 시스템을 구축하는 과정을 단계별로 다룹니다.

1.2 랭체인 메시지 객체와 모델 인터페이스

랭체인을 이용한 개발의 첫 단계는 모델과의 통신을 위한 기본 객체들을 이해하는 것입니다. 기존의 OpenAI API 사용 방식에서는 딕셔너리 리스트를 통해 대화 내용을 관리했으나, 랭체인은 이를 명시적인 클래스 객체로 변환하여 관리합니다.

에이전트의 언어: 구조화된 메시지 객체

LangChain은 LLM과의 통신을 위해 명시적인 객체를 사용합니다. 이는 코드의 안정성을 높이고 대화의 역할을 명확하게 구분하여 디버깅과 관리를 용이하게 합니다.



``SystemMessage``

AI의 페르소나와 행동 지침을 정의합니다. 대화의 가장 상위 레벨의 지시사항으로, "너는 친절한 상담사야"와 같은 역할을 부여합니다.



``HumanMessage``

사용자의 질문이나 명령을 담는 객체입니다. 실제 사용자의 입력을 래핑하여 모델에 전달합니다.



``AIMessage``

모델의 응답을 담는 객체입니다. 단순 텍스트(``content``) 외에도 토큰 사용량, 중단 이유 등 풍부한 메타데이터를 포함합니다.

1.2.1 핵심 메시지 클래스의 구조

랭체인에서 정의하는 메시지 객체는 크게 세 가지로 분류됩니다. 이는 LLM 이 대화를 이해하는 기본 단위인 '역할(Role)' 개념을 객체지향적으로 구현한 것입니다.

1. **SystemMessage:** 대화의 맥락과 AI의 페르소나(Persona)를 정의하는 메시지입니다. OpenAI API의 system 역할에 해당하며, 에이전트가 어떤 행동 지침을 따라야 하는지 설정하는 가장 상위 레벨의 지시사항을 담습니다. 예를 들어, "너는 사용자를 도와주는 친절한 상담사야"와 같은 설정이 여기에 포함됩니다.
2. **HumanMessage:** 사용자가 입력하는 질문이나 명령을 담는 객체입니다. user 역할에 해당하며, 실제 애플리케이션에서는 사용자의 키보드 입력이나 음성 인식 텍스트가 이 객체로 래핑되어 모델에 전달됩니다.
3. **AIMessage:** AI 모델이 생성한 응답을 담는 객체입니다. assistant 역할에 해당합니다. 랭체인을 통해 모델을 호출(involve)하면, 반환값은 단순한 문자열이 아니라 이 AIMessage 객체 형태로 전달됩니다. 이 객체 내부에는 실제 답변 내용인 content 뿐만 아니라, 토큰 사용량(usage_metadata), 중단 이유(finish_reason) 등 다양한 메타데이터가 포함되어 있어 디버깅과 모니터링에 유용합니다.

1.2.2 ChatOpenAI 클래스를 통한 모델 초기화

실습 환경에서는 langchain-openai 라이브러리를 통해 OpenAI 의 모델을 로드합니다. ChatOpenAI 클래스는 모델명(예: gpt-4o-mini)을 파라미터로 받아 인스턴스를 생성하며, 이 인스턴스는 랭체인(Runnable)의 표준 Runnable 인터페이스를 따릅니다. 이는 뒤에서 다룰 LCEL(LangChain Expression Language) 체인에서 이 모델을 다른 컴포넌트와 쉽게 연결할 수 있음을 의미합니다.¹

코드 레벨 분석: langchain_chatbot.ipynb

```
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4o-mini")
```

```
from langchain_core.messages import HumanMessage
model.invoke([HumanMessage(content="안녕? 나는 해피데이야.")])
```

```
model.invoke([HumanMessage(content="내 이름이 뭐지?")])
```

위 코드는 가장 기본적인 형태의 LLM 호출을 보여줍니다. 그러나 이 상태의 모델은 '기억(Memory)'을 가지고 있지 않습니다. 사용자가 "내 이름이 뭐지?"라고 후속 질문을 하더라도, 모델은 이전의 대화 내용을 기억하지 못하고 새로운 독립적인 요청으로 처리하게 됩니다. 이는 LLM 의 본질적인 특성인 'Stateless(상태 없음)'에 기인합니다.

첫 단계: 기억이 없는 Stateless 모델

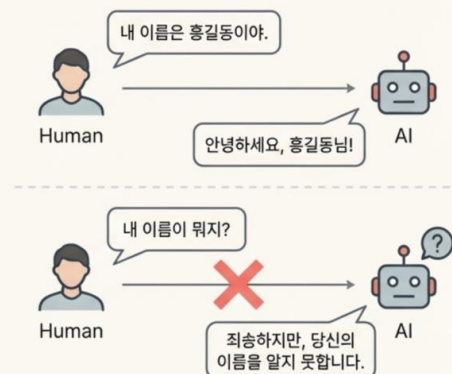
LangChain 모델의 기본 호출은 매우 간단하지만, 각 호출은 완전히 독립적입니다. 모델은 이전 대화 내용을 전혀 기억하지 못하는 'Stateless' 상태입니다.

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

# 모델 인스턴스 생성
model = ChatOpenAI(model="gpt-4o-mini")

# 첫 번째 질문
model.invoke([HumanMessage(content="내 이름은 홍길동이야.")])
# >> AIMessage(content="안녕하세요, 홍길동님!")

# 두 번째 질문 (이전 대화와 무관하게 처리됨)
model.invoke([HumanMessage(content="내 이름이 뭐지?")])
# >> AIMessage(content="죄송하지만, 당신의 이름을 알지 못합니다.")
```



진정한 대화형 AI를 구축하기 위해서는 '상태(State)'를 관리하는 메커니즘이 반드시 필요합니다.

1.3 멀티턴(Multi-turn) 대화와 상태 관리 시스템

진정한 의미의 챗봇이나 에이전트를 구현하기 위해서는 단발성 질의응답(Single-turn)이 아닌, 이전 대화의 맥락을 유지하며 이어가는 멀티턴(Multi-turn) 대화 기능이 필수적입니다. 랭체인은 이를 위해 '메시지 히스토리(Message History)'라는 추상화 계층을 제공합니다.

1.3.1 수동 히스토리 관리의 한계와 RunnableWithMessageHistory

업그레이드 #1: 대화의 맥락을 기억하는 '메모리' 장착

'RunnableWithMessageHistory'는 상태 관리 로직을 자동화하는 강력한 래퍼(Wrapper)입니다. 세션 ID를 기준으로 대화 기록을 자동으로 저장하고 로드하여, 개발자는 반복적인 작업에서 해방됩니다.

수동 리스트 관리



- 구현 복잡도: 초기엔 간단, 기능 추가 시 급증
- 세션 관리: 직접 구현 필요
- 데이터 지속성: 메모리 내 데이터 소실
- 확장성: 고급 기능 구현 어려움

RunnableWithMessageHistory



VS.

- 구현 복잡도: 초기 설정 필요, 장기적 관리 용이
- 세션 관리: 'session_id'만으로 자동 분리/관리
- 데이터 지속성: DB 연동으로 영구 저장 가능
- 확장성: 다양한 메모리 전략 적용 용이

이 구조는 'InMemoryChatMessageHistory'를 Redis, PostgreSQL 등 프로덕션용 저장소로 쉽게 교체할 수 있어 확장성이 뛰어납니다.

초기 개발 단계에서는 개발자가 직접 리스트(messages =)를 선언하고, 사용자의 입력과 AI의 출력을 매번 append 하여 관리하는 방식을 사용할 수 있습니다. 그러나 이 방식은 코드가 복잡해지고, 대화가 길어질수록 컨텍스트 윈도우(Context Window) 관리가 어려워지며, 다중 사용자 세션을 처리하기 위해 별도의 세션 관리 로직을 구현해야 하는 부담이 있습니다.

랭체인은 RunnableWithMessageHistory 클래스를 통해 이러한 상태 관리 로직을 자동화합니다. 이 클래스는 기본 체인(Chain)이나 모델(Runnable)을 감싸는 래퍼(Wrapper) 역할을 하며, 세션 ID(session_id)를 기준으로 대화 기록을 자동으로 로드하고 저장합니다.

작동 원리 분석:

1. **세션 식별:** 클라이언트가 요청을 보낼 때 config 딕셔너리에 session_id를 포함하여 전송합니다.
2. **히스토리 로드:** RunnableWithMessageHistory는 get_session_history 함수를 호출하여 해당 세션 ID에 매핑된 대화 기록 객체(예: InMemoryChatMessageHistory)를 가져옵니다.
3. **메시지 주입:** 가져온 과거 대화 기록에 현재 사용자의 메시지를 추가하여 모델에 전달합니다.
4. **자동 저장:** 모델이 응답을 생성하면, 이 응답 또한 자동으로 히스토리 객체에 저장되어 다음 턴을 대비합니다.

이러한 구조는 개발자가 대화의 '저장'과 '로드'라는 반복적인 작업에서 해방되어, 에이전트의 비즈니스 로직 구현에 집중할 수 있게 합니다. 특히 InMemoryChatMessageHistory 외에도 Redis, PostgreSQL 등 다양한 백엔드 저장소를 연결할 수 있어 프로덕션 환경으로의 확장이 용이합니다.

표 1.2: 상태 관리 방식에 따른 아키텍처 비교

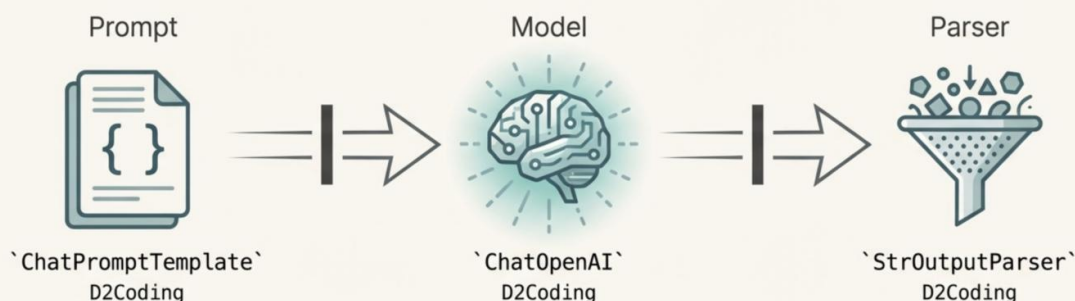
구분	수동 리스트 관리 (Manual List)	랭체인 히스토리 (RunnableWithMessageHistory)
구현 복잡도	초기에는 간단하나 기능 추가 시 급격히 증가함.	초기 설정이 필요하나 장기적으로 관리가 용이함.
세션 관리	다중 사용자 처리를 위한 딕셔너리 로직 직접 구현 필요.	session_id 파라미터만으로 다중 세션 자동 분리 및 관리.
데이터 지속성	메모리 내 리스트이므로 프로그램 종료 시 데이터 소실.	DB 연동 모듈 교체만으로 영구 저장소(Persistence) 구현 가능.
확장성	컨텍스트 트리밍 등 고급 기능 구현이 어려움.	다양한 메모리 전략(Window, Summary 등) 적용이 용이함.

제 2 강: LCEL(LangChain Expression Language)을 통한 체인 설계

2.1 LCEL 의 철학: 선언적 프로그래밍과 파이프라인

AI 로직의 청사진: LangChain 표현 언어 (LCEL)

LCEL은 복잡한 AI 워크플로우를 리눅스 파이프(|)처럼 직관적으로 연결하는 선언적 프로그래밍 방식입니다. 이를 통해 코드 가독성이 향상되고, 스트리밍, 비동기, 배치 처리와 같은 고급 기능이 자동으로 지원됩니다.



모든 컴포넌트(프롬프트, 모델, 파서)를 하나의 실행 가능한 객체, 즉 'Runnable'로 통합하여 파이프라인을 구축합니다.

랭체인은 기존의 객체지향적 체인 구성 방식에서 벗어나, 함수형 프로그래밍의 파이프라인 개념을 차용한 LCEL(LangChain Expression Language)을 도입했습니다. LCEL 은 복잡한 AI 워크플로우를 리눅스의 파이프(|) 연산자처럼 직관적으로 연결하여 구성할 수 있게 해주는 문법적 설탕(Syntactic Sugar)이자 강력한 인터페이스입니다.

LCEL 을 사용하면 프롬프트 템플릿, 모델, 출력 파서(Output Parser) 등 서로 다른 컴포넌트들을 하나의 단일한 'Runnable' 객체로 통합할 수 있습니다. 이는 코드의 가독성을 높일 뿐만 아니라, 스트리밍(Streaming), 비동기 처리(Async), 배치 처리(Batching)와 같은 고급 기능을 별도의 구현 없이 자동으로 지원하게 합니다.

2.2 프롬프트 템플릿과 변수 바인딩

LLM 에게 단순한 문자열을 전달하는 것을 넘어, 동적으로 내용이 바뀌는 템플릿을 사용해야 할 때 ChatPromptTemplate 이 사용됩니다. 이는 파이썬의 f-string 과 유사하지만, 대화형 모델에 특화된 구조화된 메시지를 생성한다는 점에서 차이가 있습니다.

예제에서는 '미녀와 야수' 이야기를 기반으로 캐릭터 페르소나를 설정하는 실습을 진행합니다. 여기서 system_template 과 human_template 을 정의하고, {story}, {character_a}, {character_b}와 같은 변수(Placeholder)를 심어둡니다.

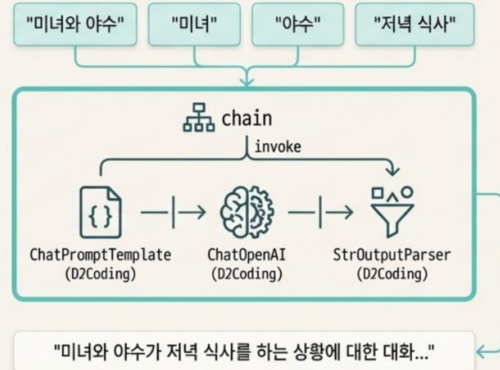
LCEL 실제 구현: 동적 프롬프트 체인 만들기

`ChatPromptTemplate`을 사용해 변수가 포함된 프롬프트를 정의하고, LCEL 체인을 통해 이를 모델 및 출력 파서와 연결합니다. `invoke` 메서드에 딕셔너리를 전달하면 변수가 동적으로 주입됩니다.

```
prompt = ChatPromptTemplate.from_messages([...]) # "미녀와 야수" 템플릿
model = ChatOpenAI(model="gpt-4o-mini")
parser = StrOutputParser()

# LCEL을 사용한 체인 구성
chain = prompt | model | parser

# 변수를 주입하여 체인 실행
response = chain.invoke({
    "story": "미녀와 야수",
    "character_a": "미녀",
    "character_b": "야수",
    "activity": "저녁 식사"
})
print(response) # >> "미녀와 야수가 저녁 식사를 하는 상황에 대한 대화..."
```



출력 파서(StrOutputParser)는 `AIMessage` 객체에서 순수 텍스트(`content`)만 추출하여 애플리케이션에서 사용하기 쉬운 형태로 만들어 줍니다.

LCEL 체인 구성 예시: sec02/lcel.ipynb

```
prompt_template = ChatPromptTemplate.from_messages([...])
model = ChatOpenAI(model="gpt-4o-mini")
parser = StrOutputParser()
```

```
# 체인 구성: 프롬프트 -> 모델 -> 파서
chain = prompt_template | model | parser
```

```
# 체인 실행 (변수 주입)
chain.invoke({
    "story": "미녀와 야수",
    "character_a": "미녀",
    "character_b": "야수",
    "activity": "저녁"
})
```

위 코드에서 chain 객체는 invoke 메서드가 호출될 때 딕셔너리 형태의 입력을 받아 프롬프트의 빈칸을 채우고, 완성된 메시지를 모델에 전달하며, 모델의 출력을 다시 파서에게 넘기는 일련의 과정을 자동으로 수행합니다. 이러한 선언적 방식은 프롬프트 엔지니어링과 모델 튜닝 과정을 분리하여 개발 효율성을 극대화합니다.

2.3 출력 파서(Output Parser)의 역할

LLM의 출력은 기본적으로 비정형 텍스트입니다. 그러나 애플리케이션 로직에서는 이를 문자열, JSON, 리스트 등 특정 데이터 타입으로 변환하여 사용해야 하는 경우가 많습니다.

StrOutputParser는 가장 기본적인 파서로, AIMessage 객체에서 content 속성만을 추출하여 순수한 문자열로 변환해줍니다.

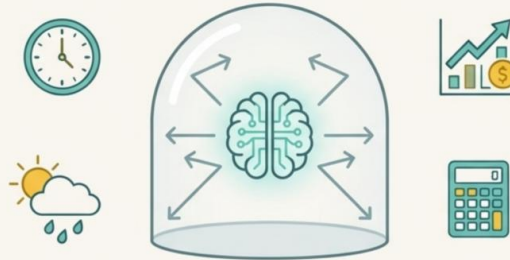
랭체인은 이 외에도 JsonOutputParser, PydanticOutputParser 등을 제공하여, LLM이 생성한 텍스트를 구조화된 데이터로 즉시 변환하고 검증할 수 있는 기능을 제공합니다. 특히 LCEL 체인 끝에 파서를 연결하면, 에이전트의 최종 응답이 항상 예측 가능한 형식을 갖추도록 강제할 수 있습니다.

제 3 강: 도구(Tools)의 도입과 에이전트의 확장

3.1 랭체인 도구(Tool) 시스템의 이해

새로운 한계: LLM은 세상과 단절되어 있다

지금까지 구축한 챗봇은 LLM이 사전에 학습한 데이터에만 의존합니다.
이는 실시간 정보나 외부 시스템과의 연동이 불가능함을 의미합니다.



오늘 서울 날씨 어때? → **Outdated Information**
256의 제곱근은? → **Potential for Calculation Errors**
삼성전자 현재 주가는? → **No Access to Real-time Data**

지금까지의 챗봇은 LLM 이 사전에 학습한 데이터에만 의존하여 답변을 생성했습니다. 그러나 날씨 정보, 주식 가격, 실시간 뉴스 등 시시각각 변하는 데이터나, 복잡한 수학 연산이 필요한 경우에는 LLM 단독으로는 정확한 답변을 제공할 수 없습니다. 이를 해결하기 위해 '도구(Tool)'라는 개념이 도입됩니다.

랭체인에서 도구란 에이전트가 외부 세계와 상호작용할 수 있도록 연결해주는 인터페이스, 즉 함수(Function)입니다. 에이전트는 사용자의 질문을 분석하여 자신이 가지고 있는 도구 중 어떤 것을 사용해야 할지 스스로 판단하고, 필요한 매개변수(Arguments)를 생성하여 도구를 호출합니다. 이는 OpenAI 의 'Function Calling' 기능을 랭체인 스타일로 추상화한 것입니다.

3.2 @tool 데코레이터를 이용한 사용자 정의 함수

업그레이드 #2: 외부 세계와 소통하는 '도구' 장착

'도구(Tool)'는 에이전트가 호출할 수 있는 함수입니다. `@tool` 데코레이터는 일반 Python 함수를 랭체인 도구로 변환하며, 이때 함수의 Docstring은 LLM이 도구의 용도와 사용법을 이해하는 '프롬프트' 역할을 합니다.

```
@tool
def get_current_time(timezone: str, location: str) -> str:
    """현재 시각을 반환하는 함수.

    Args:
        timezone (str): 'Asia/Seoul'과 같은 실제 타임존.
        location (str): 타임존과 매칭되는 도시 이름.
    """
    # ... (implementation details) ...
    return f"{location}의 현재 시간은 ... 입니다."
```

• Python 함수를 랭체인 도구로 변환

• LLM을 위한 사용 설명서 (프롬프트)

Docstring은 단순한 주석이 아닙니다. LLM은 이 설명을 분석하여 '사용자가 부산의 시간을 물어봤으니, 'timezone'은 'Asia/Seoul'로, 'location'은 '부산'으로 채워 이 함수를 호출해야겠다'고 스스로 추론합니다.

파이썬 함수를 랭체인이 인식할 수 있는 도구로 변환하기 위해 `@tool` 데코레이터를 사용합니다. 이 데코레이터는 함수의 이름, 매개변수 타입, 그리고 가장 중요한 `**Docstring(함수 설명)**`을 분석하여 LLM에게 제공할 메타데이터를 생성합니다.

실습 분석: `get_current_time` 함수

강의 자료에서는 현재 시각을 반환하는 `get_current_time` 함수를 예로 듭니다. 이 함수는 `timezone` 과 `location` 이라는 두 개의 문자열 인자를 받습니다.

코드 예시: 01_langchain_tool.ipynb

```
@tool
def get_current_time(timezone: str, location: str) -> str:
    """
    현재 시각을 반환하는 함수.

    Args:
        timezone (str): 타임존 (예: 'Asia/Seoul'). 실제 존재해야 함.
        location (str): 지역명. 타임존 코드와 매칭되는 도시 이름.
    """
    #... (구현 내용)...
```

여기서 Docstring 은 단순한 주석이 아닙니다. 이것은 LLM 에게 보내는 '프롬프트'의 일부가 됩니다. LLM 은 이 설명을 읽고 "사용자가 부산의 시간을 물어봤으니, timezone 은 'Asia/Seoul'로, location 은 '부산'으로 채워서 이 함수를 호출해야겠다"라고 추론하게 됩니다.¹ 만약 Docstring 이 부실하면 LLM 은 도구를 언제, 어떻게 사용해야 할지 판단하지 못해 오동작할 확률이 높아집니다.

3.3 Pydantic 을 활용한 입력 데이터 검증

더 똑똑하고 안전한 도구 만들기: Pydantic으로 입력 검증

복잡한 데이터를 입력받거나 엄격한 형식이 필요할 때 Pydantic 모델을 사용하면 데이터의 유효성을 검증하고 LLM에게 더 명확한 스키마를 제공할 수 있습니다.

일반 함수 기반 도구 (@tool)



구현 난이도: 낮음 (Low)

입력 복잡도: 단순 인자에 적합

유효성 검사: 기본 타입 힌트 수준

LLM 이해도: Docstring 의존도 높음

Pydantic 모델 기반 도구



구현 난이도: 중간 (Medium)

입력 복잡도: 복합 데이터 처리에 필수적

유효성 검사: 정교한 검증 가능

LLM 이해도: 필드별 description으로 명확한 지시 가능

LLM이 잘못된 형식의 인자를 생성하면 Pydantic이 오류를 발생시키고, 이 오류 메시지가 LLM에게 피드백되어 스스로 인자를 수정 후 재시도하게 만드는 메커니즘을 지원합니다.

단순한 데이터 타입(str, int)을 넘어 복잡한 구조의 데이터를 입력받거나, 데이터의 형식을 엄격하게 검증해야 할 때는 Pydantic 라이브러리를 활용합니다. Pydantic 은 데이터 파싱 및 유효성 검사 라이브러리로, 랭체인과 깊게 통합되어 있습니다.

강의 자료의 get_yf_stock_history 함수 예제에서는 주식 티커(Ticker)와 기간(Period)을 입력받는데, 이를 StockHistoryInput 이라는 Pydantic 모델로 정의하여 관리합니다.

구조적 검증의 이점:

- 명확한 스키마 정의:** BaseModel 과 Field 를 사용하여 각 필드의 의미와 제약 조건을 명시합니다. 이는 LLM 에게 더 강력한 힌트를 제공합니다.
- 오류 방지:** LLM 이 잘못된 형식의 데이터를 생성하려 할 경우, Pydantic 단계에서 유효성 검사 에러가 발생합니다. 랭체인은 이 에러 메시지를 다시 LLM 에게 피드백하여, LLM 이 스스로 인자값을 수정하여 재시도하게 만드는 메커니즘을 지원합니다.

표 3.1: 일반 함수와 Pydantic 모델 기반 도구의 비교

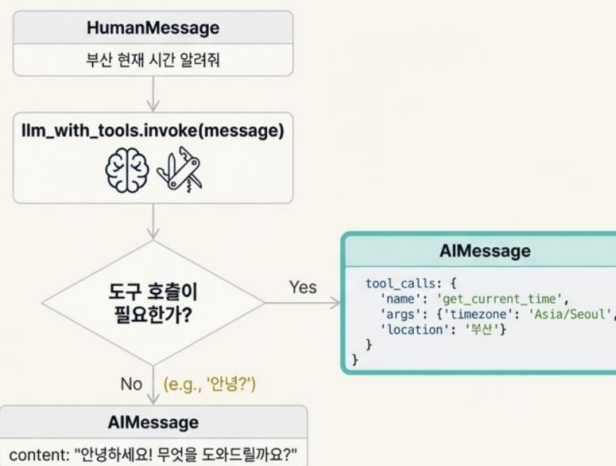
특성	일반 함수 기반 도구 (@tool)	Pydantic 모델 기반 도구
구현 난이도	낮음 (데코레이터만 사용).	중간 (클래스 정의 필요).
입력 복잡도	단순한 인자 처리에 적합.	중첩된 구조나 복합 데이터 처리에 필수적.
유효성 검사	기본적인 타입 힌트 수준.	정규표현식, 값 범위 등 정교한 검증 가능.
LLM 이해도	Docstring 의존도가 높음.	필드별 description 을 통해 명확한 지시 가능.

코드예시 : 02_pydantic.ipynb

3.4 모델과 도구의 바인딩 (bind_tools)

모델과 도구 연결하기: `.bind_tools()`

`llm.bind_tools(tools)` 메서드는 정의된 도구들을 LLM이 이해할 수 있는 JSON 스키마로 변환하여 모델에 주입합니다. 이후 모델은 사용자의 질문을 분석하여 도구 호출이 필요한지 스스로 판단합니다.



정의된 도구들을 LLM 이 사용할 수 있도록 연결하는 과정이 필요합니다. `llm.bind_tools(tools)` 메서드는 랭체인의 도구 객체들을 모델이 이해할 수 있는 JSON 스키마 포맷으로 변환하여 모델에 주입합니다.

바인딩된 모델(`llm_with_tools`)을 호출하면, 모델은 바로 텍스트 응답을 내놓는 대신 **도구 호출이 필요한지 여부**를 판단합니다. 도구 호출이 필요하다고 판단되면 `AIMessage` 객체의 `tool_calls` 속성에 호출할 함수의 이름과 인자값을 담아 반환합니다. 반면, 일상적인 대화(예: "안녕?")의 경우 도구 호출 없이 일반적인 텍스트 응답을 반환합니다.

제 4 강: 스트림(Stream) 방식의 데이터 처리와 사용자 경험 최적화

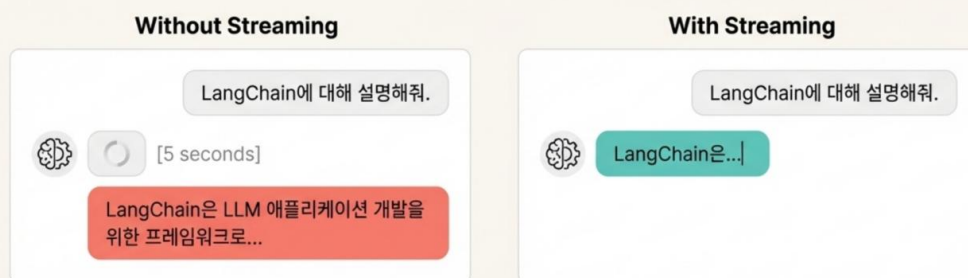
4.1 스트리밍의 필요성과 원리

사용자 경험 최적화: 실시간 응답을 위한 스트리밍

LLM의 응답 생성 시간을 기다리게 하는 것은 좋지 않은 UX를 만듭니다.

스트리밍은 생성되는 토큰을 실시간으로 전송하여 사용자가 즉각적인 피드백을 받게 합니다.

LangChain에서는 `.invoke()` 대신 `.stream()` 메서드를 사용합니다.



`.stream()`은 완성된 `AIMessage`가 아닌, `AIMessageChunk` 객체들을 생성하는 이터레이터(Iterator)를 반환합니다. 개발자는 반복문을 통해 청크를 순차적으로 처리해야 합니다.

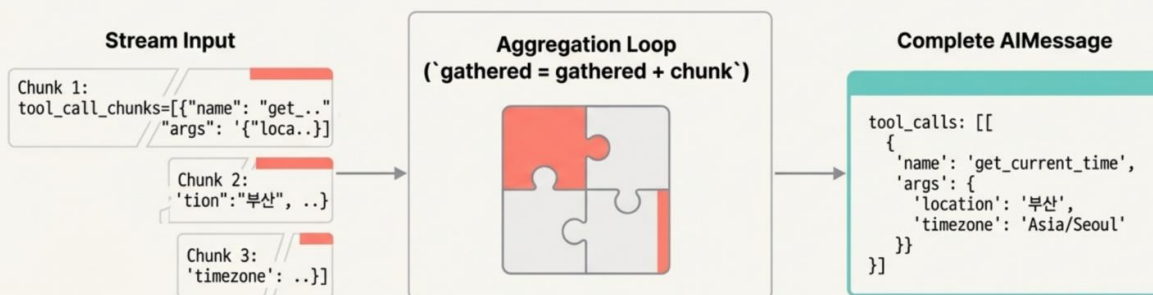
LLM은 응답을 생성하는 데 일정 시간이 소요됩니다. 특히 긴 문장을 생성하거나 복잡한 추론을 할 때 사용자가 빈 화면을 보고 기다리게 하는 것은 좋지 않은 사용자 경험(UX)을 제공합니다. 이를 해결하기 위해 생성되는 토큰을 실시간으로 전송하는 '스트리밍(Streaming)' 기술이 사용됩니다. 랭체인에서는 `.invoke()` 대신 `.stream()` 메서드를 사용하여 스트리밍을 활성화합니다. 이 경우 반환값은 완성된 `AIMessage` 객체가 아니라, `AIMessageChunk` 객체들을 생성하는 이터레이터(Iterator)가 됩니다. 개발자는 반복문(for)을 통해 이 청크들을 받아 순차적으로 처리해야 합니다.

코드 예시 : `sec04/langchain_tool_stream.ipynb`

4.2 도구 호출 시의 스트리밍 처리 전략

스트리밍 심화: 도구 호출 청크(Chunk) 처리하기

도구 호출이 포함된 스트림에서는, 도구 이름과 인자(JSON) 역시 여러 청크로 나뉘어 전송됩니다. 불완전한 JSON 조각은 실행할 수 없으므로, 모든 `tool\_call\_chunks`를 하나로 합친 후 도구를 실행해야 합니다.



랭체인의 `AIMessageChunk`는 덧셈 연산자(+)가 오버로딩되어 있어, 청크들을 더하기만 하면 텍스트 내용과 `tool\_call\_chunks` 데이터가 지능적으로 병합됩니다. 스트림이 종료된 후, 완성된 객체에서 도구를 실행합니다.

일반 텍스트 응답의 스트리밍은 단순히 문자열을 이어 붙여 출력하면 되지만, 도구 호출이 포함된 경우 상황은 복잡해 집니다. 모델이 도구를 호출하기로 결정했을 때, 도구의 이름과 인자(JSON 문자열) 또한 스트림 형태로 쪼개져서 전송되기 때문입니다.

불완전한 JSON 조각(예: {"loc...)은 파싱 할 수 없으므로, 도구를 실행하기 위해서는 스트림으로 들어오는 모든 tool_call_chunks 를 모아서(Aggregate) 완전한 형태의 메시지로 조립해야 합니다.

코드 구현 패턴 분석:

강의 자료의 langchain_tool.ipynb 실습에서는 gathered 라는 변수를 사용하여 청크를 누적합니다.

1. **청크 수신:** 스트림을 통해 chunk 가 들어옵니다.
2. **누적 연산:** gathered = gathered + chunk (또는 +=) 연산을 수행합니다. 랭체인 AIMessageChunk 클래스는 덧셈 연산자가 오버로딩되어 있어, 텍스트 내용은 물론 내부의 tool_call_chunks 데이터까지 지능적으로 병합해 줍니다.
3. **완료 후 실행:** 스트림이 종료되면 완성된 gathered 객체에서 tool_calls 정보를 확인하고, 해당 도구를 실행합니다.

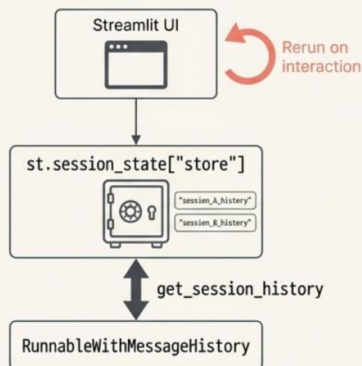
이러한 패턴은 비동기적이고 파편화된 데이터 흐름을 동기적이고 완결된 실행 단위로 변환하는 핵심적인 기술입니다.

제 5 강: Streamlit 을 활용한 웹 인터페이스 구현 및 실전 배포

5.1 Streamlit 의 실행 모델과 세션 상태(Session State)

최종 구현: Streamlit으로 웹 인터페이스 구축하기

Streamlit은 사용자의 모든 상호작용마다 스크립트전체를 재실행합니다. 이로 인해 일반 변수에 저장된 대화 기록은 초기화됩니다. 이를 방지하기 위해 반드시 **st.session_state**를 사용하여 대화 히스토리를 연속적으로 관리해야 합니다.



사용자에게 실시간 응답을 보여주기 위해, LangChain의 스트림을 `st.write_stream`(텍스트 전용) 또는 `st.empty()`를 활용한 커스텀 루프 (도구 호출 포함)와 연결합니다.

Streamlit 은 파이썬만으로 빠르게 웹 애플리케이션을 구축할 수 있는 도구이지만, 그 실행 모델을 정확히 이해해야 랭체인과 연동할 수 있습니다. Streamlit 은 사용자의 상호작용이 있을 때마다 스크립트 전체를 처음부터 다시 실행(Rerun)하는 방식을 취합니다.

이러한 특성 때문에, 단순히 변수에 대화 기록을 저장하면 화면이 새로고침될 때마다 대화 내용이 초기화되는 문제가 발생합니다. 이를 해결하기 위해 `st.session_state` 라는 영속적인 저장소를 사용해야 합니다.

코드 예시 : 01_langchain_simple_chat_streamlit.py

세션 상태 관리 전략:

- `st.session_state["messages"]`: 화면에 표시할 채팅 메시지 리스트를 저장합니다.
- `st.session_state["store"]`: 랭체인이 `RunnableWithMessageHistory` 가 참조할 실제 대화 히스토리 객체를 저장합니다. 세션 ID 별로 구분된 `InMemoryChatMessageHistory` 객체들이 이곳에 보관됩니다.

5.2 랭체인 메모리와 Streamlit 의 연동 구현

langchain_simple_chat_streamlit.py 실습에서는 랭체인 메모리 시스템을 Streamlit 환경에 이식하는 과정을 보여줍니다. 핵심은 `get_session_history` 함수를 정의할 때, 새로운 히스토리 객체를 생성하는 것이 아니라 `st.session_state["store"]`를 조회하도록 만드는 것입니다.

```
def get_session_history(session_id: str):
    if session_id not in st.session_state["store"]:
        st.session_state["store"][session_id] = InMemoryChatMessageHistory()
    return st.session_state["store"][session_id]
```

이렇게 연결하면 Streamlit 이 화면을 다시 그려도(Rerun), 랭체인 모델은 `st.session_state` 에 보존된 과거 대화 기록을 참조하여 맥락을 유지할 수 있습니다.

5.3 UI 스트리밍 구현: `st.write_stream` 과 커스텀 루프

사용자에게 실시간으로 답변이 생성되는 모습을 보여주기 위해 Streamlit 의 UI 컴포넌트에 랭체인 스트림을 연결해야 합니다.

1. **st.write_stream**: 최신 Streamlit 버전에 추가된 기능으로, 제너레이터(Generator)를 입력으로 받아 타자기 효과를 자동으로 구현해 줍니다. 텍스트 전용 스트리밍에 매우 적합하고 구현이 간단합니다.
2. **커스텀 루프 (st.empty)**: 도구 호출과 같이 복잡한 로직이 포함된 경우, `st.write_stream` 만으로는 부족할 수 있습니다. 이 경우 `with st.chat_message("assistant")`: 블록 내부에서 `st.empty()`로 빈 컨테이너를 만들고, 랭체인 스트림에서 청크를 받을 때마다 컨테이너의 내용을 업데이트(write 또는 markdown)하는 방식을 사용합니다.

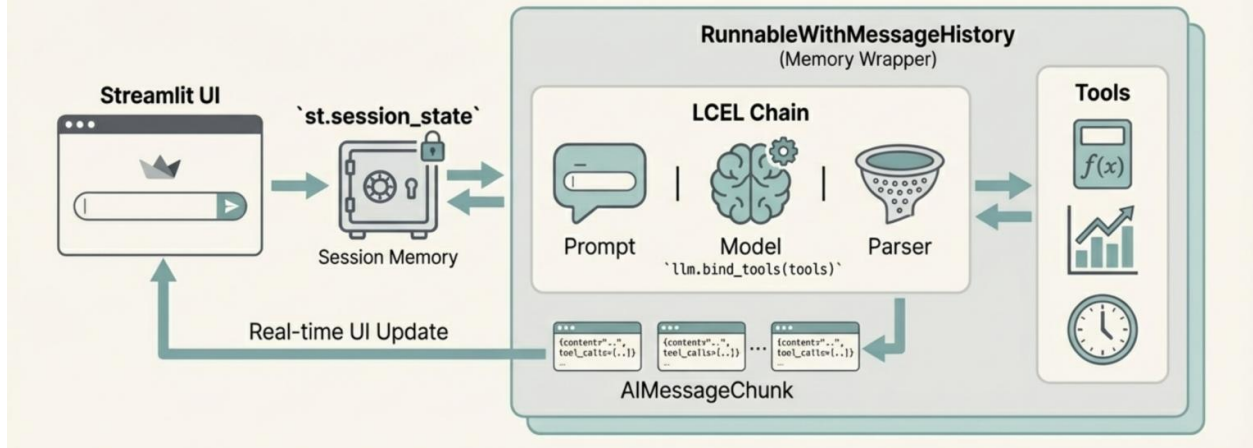
실전 코드 패턴:

```
# 도구 실행 결과를 포함한 복합 스트리밍 처리
response = llm_with_tools.stream(messages)
with st.chat_message("assistant").empty():
    for chunk in response:
        # 텍스트 청크와 도구 청크를 구분하여 처리 및 UI 업데이트
        #... (생략)...
```

이 방식은 도구가 실행되는 동안 "분석 중..."과 같은 중간 상태를 보여주거나, 도구 실행 결과를 별도의 UI 요소(표, 그래프 등)로 렌더링하는 등 고도화된 UX 를 구현할 수 있게 해줍니다.

완성된 에이전트 아키텍처

우리는 단순한 LLM 호출에서 시작하여 메모리, LCEL, 도구, 스트리밍, UI를 단계적으로 통합하여 지능형 에이전트 시스템을 완성했습니다.



5.4 랭체인 메모리 없이 멀티턴 구현하기 (심화)

코드 예시 : 02_langchain_streamlit_tool.py

때로는 랭체인 `RunnableWithMessageHistory` 추상화를 건너내고, 개발자가 직접 리스트를 관리해야 할 때가 있습니다. 예를 들어, 대화 기록을 데이터베이스에 커스텀 스키마로 저장하거나, 특정 메시지를 수정/삭제해야 하는 경우입니다.

`langchain_streamlit_tool.py`에서는 `st.session_state["messages"]` 리스트를 랭체인 모델에 직접 주입하여 멀티턴을 구현합니다. 이 경우, 개발자는 모델의 응답(`AIMessage`)뿐만 아니라, 도구 실행 결과(`ToolMessage`)까지 정확한 순서대로 리스트에 `append` 해야 합니다. 만약 순서가 꼬이거나 `ToolMessage`가 누락되면, LLM은 "도구를 호출했는데 결과가 오지 않았다"고 판단하여 오류를 일으키거나 환각(Hallucination) 증세를 보일 수 있습니다.

결론 및 제언: 에이전트 시스템의 미래

랭체인을 활용하여 단순한 챗봇을 넘어선, 도구를 활용하고 상태를 관리하며 사용자와 실시간으로 상호작용하는 에이전트 시스템을 구축하는 전 과정을 살펴보았습니다.

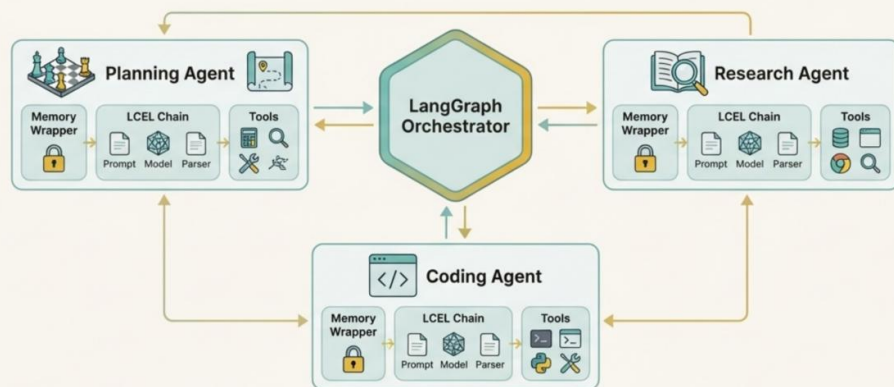
핵심 요약:

1. **아키텍처의 전환:** 직접적인 API 호출에서 랭체인이라는 오케스트레이션 계층을 도입함으로써 유연성과 확장성을 확보했습니다.
2. **LCEL의 도입:** 선언적 파이프라인 구성을 통해 코드의 가독성을 높이고 스트리밍 등의 기능을 내재화했습니다.
3. **도구의 지능화:** Pydantic 과 @tool 데코레이터를 통해 LLM 이 외부 세계와 안전하고 정확하게 소통할 수 있는 인터페이스를 구축했습니다.
4. **사용자 경험의 고도화:** Streamlit 의 세션 상태 관리와 스트리밍 UI 렌더링 기술을 접목하여 상용 수준의 애플리케이션을 구현했습니다.

향후 에이전트 기술은 단일 에이전트를 넘어, 여러 에이전트가 협력하여 문제를 해결하는 멀티 에이전트 시스템(Multi-Agent Systems)으로 발전할 것입니다. 본 장에서 다룬 기초 기술들은 LangGraph 와 같은 상위 레벨의 프레임워크를 이해하고 활용하는 데 있어 필수적인 기반 지식이 될 것입니다.

다음 단계: 멀티 에이전트 시스템을 향하여

본 과정에서 다룬 단일 에이전트 구축 기술은 미래 AI 개발의 핵심 기반입니다. 이 지식은 여러 에이전트가 협력하여 복잡한 문제를 해결하는 '멀티 에이전트 시스템'으로 확장될 것입니다.



오늘 배운 아키텍처 원칙(모듈화, 상태 관리, 도구 사용)은 LangGraph와 같은 상위 레벨 프레임워크를 이해하고, 더 복잡하고 강력한 AI 시스템을 설계하는 데 필수적입니다.

부록: 기술 용어 및 참조 데이터

표 A.1: 랭체인 주요 컴포넌트 요약

컴포넌트	역할 및 설명	주요 클래스/함수
Model	언어 모델 래퍼. 입력과 출력을 표준화함.	ChatOpenAI, ChatAnthropic
Prompt	동적 텍스트 생성을 위한 템플릿 엔진.	ChatPromptTemplate
Output Parser	모델의 출력을 구조화된 데이터로 변환.	StrOutputParser, PydanticOutputParser
Memory	대화의 맥락을 저장하고 조회하는 모듈.	RunnableWithMessageHistory, InMemoryChatMessageHistory
Tool	에이전트가 실행할 수 있는 외부 함수.	@tool, BaseTool
Chain	컴포넌트들을 연결하여 실행 흐름을 정의.	LCEL 파이프 연산자 (‘